Propósito del módulo

MÓDULO 4

A05 y A06 — Inyección y Diseño Inseguro

OWASP Top 10:2025 | Java 17 · Jakarta EE 11 · PostgreSQL 16

50 diapositivas · Laboratorios y escenarios financieros hipotéticos



A05 en OWASP Top 10:2025

A05:2025 — Inyección ocupa el lugar #5, después de descender dos posiciones desde el #3. Reúne fallas donde datos no confiables llegan a un intérprete y alteran la estructura o el significado de una instrucción..



A05 protege la interpretación; A06 protege las decisiones de diseño.

A05:2025 · Definición operativa

Una vulnerabilidad de inyección aparece cuando una aplicación permite que entradas no confiables lleguen a un intérprete —por ejemplo, una base de datos, un navegador o una línea de comandos— y provoca que parte de esa entrada se ejecute como instrucciones.

El riesgo no está en recibir caracteres especiales por sí mismos. Aparece cuando la aplicación mezcla datos con la estructura del comando o consulta y el intérprete ya no puede distinguir qué parte es dato y qué parte es instrucción.

El control fundamental consiste en mantener separados los datos y los comandos, preferentemente mediante APIs seguras y consultas parametrizadas.



CWE-74 · Neutralización inadecuada de elementos especiales.

Cómo se produce la inyección

El flujo típico inicia cuando un usuario controla un parámetro, encabezado, cookie, campo JSON, URL o archivo. La aplicación incorpora ese dato a una consulta o instrucción dinámica sin separación robusta.

El intérprete analiza el resultado completo. Si los datos modifican la sintaxis, la aplicación puede ejecutar una consulta diferente, mostrar registros no previstos, ejecutar comandos o generar contenido activo en el navegador.

El análisis debe seguir el dato desde su origen hasta el punto donde se consume. Validar únicamente el formulario no demuestra que el servidor construya instrucciones seguras.



Entrada controlada → concatenación insegura → interpretación alterada.

Intérpretes y superficies de inyección

Las formas comunes incluyen SQL, NoSQL, comandos del sistema operativo, ORM, LDAP, XML, expresiones y otras interfaces que interpretan instrucciones. El principio es equivalente aunque cambie la sintaxis.

Las entradas pueden llegar por parámetros, encabezados, rutas, cookies, JSON, SOAP, XML o datos persistidos. También pueden provenir de integraciones que la aplicación considera confiables sin verificar su contenido.

Las pruebas deben abarcar todas las superficies y no limitarse a campos visibles. Revisión de código, pruebas automatizadas y fuzzing aportan perspectivas complementarias.



No basta con probar el formulario: rastrear todos los flujos de datos.

Validación, sanitización y codificación

La validación comprueba si un dato cumple reglas de negocio y formato esperado. Debe realizarse en el servidor, incluso cuando el cliente ya valida campos, porque el cliente puede modificarse o evitarse.

La sanitización transforma o retira contenido no deseado, pero no es una defensa universal: los caracteres válidos dependen del contexto. Una regla genérica de reemplazo puede dejar variantes sin cubrir.

La codificación de salida transforma datos para que se representen como texto en un contexto concreto: HTML, atributo HTML, JavaScript o URL. Debe aplicarse en el punto de salida y según el contexto.



Validar ≠ parametrizar ≠ codificar salida.

Defensa principal: separar datos e instrucciones

La opción preferida es usar APIs que eviten invocar directamente el intérprete, proporcionen interfaces parametrizadas o abstraigan consultas mediante ORM de forma segura.

En SQL, los parámetros se envían como valores separados de la cadena de consulta. El motor interpreta la estructura fija y trata el valor recibido como dato, no como una nueva cláusula SQL.

Los nombres de tablas, columnas y otras partes estructurales no suelen poder parametrizarse como valores. Si deben elegirse dinámicamente, use un mapeo cerrado entre opciones permitidas y fragmentos predefinidos.



Primero parametrizar; no confiar en filtros de caracteres.

SQL Injection · consulta vulnerable

Una consulta es vulnerable cuando concatena directamente un parámetro no confiable dentro de una cadena SQL. El usuario puede introducir caracteres que cierren una cadena, agreguen condiciones o alteren la lógica.

En un sistema financiero ficticio, un endpoint de consulta de movimientos recibe un identificador de cuenta. Si se concatena, la consulta podría devolver registros distintos de los autorizados.

El problema está en la construcción de la consulta: PostgreSQL ejecuta la instrucción que la aplicación le entrega.



```
String sql = "SELECT * FROM accounts WHERE id = '" + input + "'";
```

La entrada puede modificar la estructura de la consulta.

SQL Injection · consulta parametrizada

Con JDBC, una sentencia preparada separa la estructura SQL de los valores. El parámetro se asigna mediante un método tipado y el motor no lo interpreta como parte de la sintaxis.

En Java, PreparedStatement permite definir la consulta con marcadores y asignar después los valores. En Jakarta EE, la persistencia debe mantener la misma separación y evitar concatenar entradas en JPQL o SQL.

La parametrización no corrige por sí sola autorización deficiente ni exposición excesiva de columnas. La consulta también debe limitarse al usuario y recurso autorizados.



```
String sql = "SELECT * FROM accounts WHERE id = ?";
PreparedStatement ps = conn.prepareStatement(sql);
ps.setString(1, accountId);
```

PreparedStatement + autorización del lado servidor.

SQL Injection · parámetros dinámicos

Filtros opcionales, ordenamientos y reportes suelen generar consultas dinámicas. Los valores de filtros pueden parametrizarse; los identificadores estructurales —columna u orden ASC/DESC— requieren selección mediante lista permitida.

El servidor puede traducir la opción `fecha` a una constante interna `created_at` y `monto` a `amount`. Cualquier otra opción se rechaza o sustituye por un valor predeterminado seguro.

Evite aceptar fragmentos SQL completos desde el navegador. Una interfaz de reportes debe ofrecer opciones limitadas y documentadas, no una consola SQL indirecta.



Allow-list para estructura dinámica; parámetros para valores.

ORM y HQL también pueden ser vulnerables

Usar Hibernate u otro ORM no garantiza automáticamente que una consulta sea segura. Si se concatena entrada del usuario en HQL, JPQL o expresiones de búsqueda, puede alterarse la consulta igual que en SQL.

OWASP presenta un escenario donde una consulta HQL concatena un identificador de cliente. La entrada puede ampliar los registros devueltos.

Use parámetros nombrados o posicionales del ORM y revise las consultas dinámicas. Compruebe además que la capa de aplicación aplica autorización y limita el conjunto de resultados.



ORM no significa inmunidad: la concatenación sigue siendo el problema.

Blind SQL Injection · concepto

En Blind SQL Injection, la aplicación no muestra directamente el resultado ni un error SQL útil. Sin embargo, las respuestas pueden revelar información indirectamente mediante diferencias de comportamiento.

El atacante puede observar respuestas booleanas —una página que cambia cuando una condición es verdadera— o diferencias de tiempo cuando una condición provoca una operación más lenta.

Que no aparezcan datos en pantalla ni errores detallados no demuestra que no exista inyección. Se requieren pruebas controladas y revisión del código que genera la consulta.



La aplicación puede filtrar por comportamiento, no solo por contenido.

Blind SQL Injection · señales observables

Las señales pueden incluir cambios en el tamaño de respuesta, mensajes genéricos distintos, diferencias de estado HTTP, variaciones repetibles de tiempo o efectos laterales en una operación.

Las pruebas deben ejecutarse en un entorno autorizado y controlado, con datos sintéticos y límites para evitar afectar disponibilidad o modificar información real.

Compare solicitudes equivalentes, documente condiciones de prueba y confirme el origen en el código o en trazas seguras. Una variación aislada de latencia no es prueba concluyente.



Comparar condiciones equivalentes y confirmar la causa en el servidor.

Cross-Site Scripting (XSS) · concepto

XSS ocurre cuando una aplicación incorpora contenido no confiable en una página y el navegador lo interpreta como código activo en lugar de texto. El impacto puede incluir acciones en nombre del usuario, lectura de datos accesibles o manipulación de interfaz.

OWASP mapea XSS como CWE-79. La categoría 2025 señala que XSS tiene alta frecuencia y un impacto ponderado menor que SQL Injection, aunque la severidad depende del contexto y de los privilegios de la víctima.

El control principal es codificar la salida según el contexto y evitar insertar contenido no confiable en sinks peligrosos.



CWE-79 · El navegador también es un intérprete.

XSS reflejado, almacenado y basado en DOM

En XSS reflejado, la entrada llega en una solicitud y vuelve a la respuesta sin codificación adecuada. La explotación suele depender de que una víctima abra un enlace o envíe una solicitud manipulada.

En XSS almacenado, el contenido se guarda —por ejemplo, comentarios, descripciones o mensajes— y se ejecuta cuando otras personas consultan el contenido.

En XSS basado en DOM, el problema ocurre en el código del cliente cuando datos controlables llegan a APIs que interpretan contenido como HTML o script.



Tres rutas distintas; el control debe cubrir servidor y cliente.

XSS · contextos de salida

Evite concatenar datos no confiables en bloques de script, atributos de eventos o fragmentos HTML. Prefiera APIs y plantillas que realicen codificación contextual automáticamente.

Cuando se permita HTML enriquecido, use un sanitizador mantenido y una política explícita de elementos y atributos permitidos. La validación de entrada no sustituye la codificación de salida.



Codificar al salir, en el contexto correcto.

Laboratorio 1 · consulta vulnerable y segura

Objetivo: Identificar los posibles lugares donde se puede inyectar XSS.

- Reflected XSS
- Stored XSS
- DOM XSS



Entregable: diff del código, prueba automatizada y explicación.

Laboratorio 2 · XSS y codificación contextual

Objetivo: Comprometer una session mediante el secuestro de la Cookie o hijacking.



Entregable: mapa fuente-sink y pruebas de salida.